

Complete Jenkins & Git Manual for College Lab

Clearing Old User Credentials, Setting Up Git Identity, Project File Creation, Git Lifecycle & 5 Separate Jenkins Pipelines

1. Removing Old Credentials & Setting Up Local Git Identity

On shared college PCs, previous users' credentials or incorrect email addresses like 'tanmaypondhe7777@gmail.com' cause conflicts. Follow these steps to clear them and set your personal details.

Step 1.1: Clear Cached Windows Credentials

1. Press **Windows Key + R**, type **control /name Microsoft.CredentialManager** and press Enter.
2. Click **Windows Credentials**.
3. Scroll down to **Generic Credentials**.
4. Click on any entry named **git:https://github.com** or **vscode-vscode.github-authentication** and click **Remove**.

Step 1.2: Remove Old/Global Git Configurations

```
# Run these in PowerShell / VS Code Terminal:
# 1. Unset any incorrect global email/username
git config --global --unset user.name
git config --global --unset user.email

# 2. Unset any incorrect local settings (if inside a git folder)
git config --local --unset user.name
git config --local --unset user.email
```

Step 1.3: Set Correct Local Git User Credentials

```
# First initialize git in your folder if not already done:
git init

# Set your current username and correct email:
git config --local user.name "tanp4577-web"
git config --local user.email "tanmaypondhe7777@gmail.com"

# Verify settings:
git config --local --list
```

2. Project Files Setup

Create the following two files inside your working directory (e.g., C:\Users\pondh\Downloads\jenkins):

File 1: addition.py

```
# addition.py
a = 10
b = 20
result = a + b

print("First Number:", a)
print("Second Number:", b)
print("Sum:", result)
```

File 2: README.md

```
# Jenkins Pipeline Demo Repository
This repository contains sample code and pipeline scripts for Jenkins CI/CD integration testing.

## Files Included:
- addition.py
- README.md
```

3. Step-by-Step Guide: How to Connect & Push to GitHub

```
# 1. Open PowerShell in your folder (C:\Users\pondh\Downloads\jenkins)

# 2. Initialize Git repository
git init

# 3. Add all files to staging
git add .

# 4. Commit files with a message
git commit -m "Initial commit: Add addition.py and README.md"

# 5. Set main branch name
git branch -M main

# 6. Connect your local folder to GitHub remote repository
git remote add origin https://github.com/tanp4577-web/jenkinspipeline2.git

# 7. Push files to GitHub
git push -u origin main
```

[GITHUB AUTHENTICATION] When Windows prompts for credentials on 'git push':

- Username: tanp4577-web
- Password: Your Personal Access Token (PAT) generated from GitHub (Developer Settings → Personal Access Tokens).

4. 5 Distinct Jenkins Pipeline Implementations

Each pipeline script below is completely separate and serves a specific build step:

Pipeline Code 1: Checkout Code & Print Directory

Objective: Checkout GitHub code and display the workspace path using `bat 'cd'`.

```
pipeline {
  agent any
  stages {
    stage('Checkout Code') {
      steps {
        git branch: 'main', url: 'https://github.com/tanp4577-
web/jenkinspipeline2.git'
      }
    }
    stage('Print Directory') {
      steps {
        bat 'cd'
      }
    }
  }
}
```

Pipeline Code 2: Checkout Code & Print Success Message

Objective: Checkout GitHub code and print a success message to the console.

```
pipeline {
  agent any
  stages {
    stage('Checkout Code') {
      steps {
        git branch: 'main', url: 'https://github.com/tanp4577-
web/jenkinspipeline2.git'
      }
    }
    stage('Print Message') {
      steps {
        echo 'Hello! Jenkins Pipeline executed successfully'
      }
    }
  }
}
```

Pipeline Code 3: Checkout Code & Execute Python Program (`addition.py`)

Objective: Checkout GitHub code and execute `addition.py` to print addition results.

```
pipeline {
    agent any
    stages {
        stage('Checkout Code') {
            steps {
                git branch: 'main', url: 'https://github.com/tanp4577-
web/jenkinspipeline2.git'
            }
        }
        stage('Execute Python Program') {
            steps {
                bat 'python addition.py'
            }
        }
    }
}
```

Pipeline Code 4: Checkout Code & Create Dynamic File

Objective: Checkout GitHub code and dynamically create a text file named `demo.txt`.

```
pipeline {
    agent any
    stages {
        stage('Checkout Code') {
            steps {
                git branch: 'main', url: 'https://github.com/tanp4577-
web/jenkinspipeline2.git'
            }
        }
        stage('Create File') {
            steps {
                bat 'echo This file is created by Jenkins > demo.txt'
            }
        }
    }
}
```

Pipeline Code 5: Checkout Code & Read File (`README.md`)

Objective: Checkout GitHub code and display contents of `README.md` using `bat 'type README.md'`.

```
pipeline {
    agent any
    stages {
        stage('Checkout Code') {
```

```

        steps {
            git branch: 'main', url: 'https://github.com/tanp4577-
web/jenkinspipeline2.git'
        }
    }
    stage('Read File') {
        steps {
            bat 'type README.md'
        }
    }
}
}

```

5. How to Run Pipelines in Jenkins Dashboard

1. Open Jenkins in browser (<http://localhost:8080>).
2. Click **New Item** → Enter job name (e.g., pipeline1) → Select **Pipeline** → Click **OK**.
3. Scroll down to **Pipeline** section and set Definition to **Pipeline script**.
4. Copy and paste any ONE of Pipeline Codes 1–5 into the box.
5. Click **Save** → Click **Build Now**.
6. Click on the build number (e.g., #1) under Build History → **Console Output** to view execution logs.